

# Java 11 HttpClient

`HttpClient` was officially introduced with Java 11. It was first introduced in Java 9 as an incubator. You could say it is a new version of `java.net.HttpURLConnection`.

It offers many new features:

- Supports both HTTP 1.1 and HTTP 2 (default)
- Supports both synchronous and asynchronous calls
- Provides reactive data and streams to both request and response with non-blocking back pressure

It works very well in asynchronous mode and with streams. However, here, we'll only cover the synchronous calls to align with other REST clients.

First, we'll add a provider class that will build the `HttpClient` and provide methods to build and send HTTP requests, as shown in the following example:

```
public class RestClient {
    HttpClient httpClient = HttpClient
        .newBuilder()
        .followRedirects(HttpClient.Redirect.NORMAL)
        .build();

    // Returns pre-configured request builder
    // after setting given parameter values
    public Builder requestBuilder(Uri uri,
        Optional<Map<String, String>> additionalHeaders) {
        Builder builder = HttpRequest.newBuilder()
            .uri(uri)
            .timeout(Duration.ofMinutes(1))
            .header("Content-Type", "application/json");
        if (additionalHeaders.isPresent()) {
            additionalHeaders.get().forEach((k, v) ->
                builder.header(k, v));
        }
        return builder;
    }

    // It calls and returns the response of given request
    public HttpResponse<String> send(HttpRequest request)
        throws IOException, InterruptedException {
        return httpClient.send(request, BodyHandlers.ofString());
    }
}
```

Now, we'll use this `RestClient` to create a REST client for user-service. We have added the CRUD method. Also, a patch method is added to demonstrate how to call other HTTP methods that's not available as an API. The `BodyPublisher` class is used to create the request body. We are using the string. You could use the byte array or streams, and so on.

Similarly, the response is also consumed as a string; you could change it to a byte array, streams, and so on. This is all shown in the following code:

```

@Component
public class UserRestClient {
    @Autowired
    private ObjectMapper objectMapper;
    private final RestClient restClient = new RestClient();

    private static final String userEndpoint = "http://localhost:2224/v1/user";

    public void getUser() throws Exception {
        HttpRequest request = restClient.requestBuilder(
            URI.create(userEndpoint + "?name=x"), Optional.empty()).GET().build();
        HttpResponse<String> response = restClient.send(request);
        if (response.statusCode() == 200) {
            objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false)
            UserVO[] userVO = objectMapper.readValue(response.body(), UserVO[].class);
        }
    }

    public void postUser() throws Exception {
        UserVO userVO = new UserVO();
        userVO.setId("3");
        userVO.setName("X User");
        userVO.setAddress("X Address");
        userVO.setCity("X City");
        userVO.setPhoneNo("1234567890");
        HttpRequest request = restClient.requestBuilder(
            URI.create(userEndpoint), Optional.empty()).POST(BodyPublishers.ofString(
                objectMapper.writeValueAsString(userVO))).build();
        HttpResponse<String> response = restClient.send(request);
    }

    public void putUser() throws Exception {
        UserVO userVO = new UserVO();
        userVO.setId("3");
        userVO.setName("X User 1");
        userVO.setAddress("X Address 1");
        userVO.setCity("X City 1");
        userVO.setPhoneNo("1234567899");
        HttpRequest request = restClient.requestBuilder(
            URI.create(userEndpoint + "/3"), Optional.empty()).PUT(BodyPublishers.ofString(
                objectMapper.writeValueAsString(userVO))).build();
        HttpResponse<String> response = restClient.send(request);
    }

    public void patchUser() throws Exception {
        HttpRequest request = restClient.requestBuilder(
            URI.create(userEndpoint + "/3/name?value=Duke+Williams"),
            Optional.empty()).method("PATCH", BodyPublishers.noBody()).build();
        HttpResponse<String> response = restClient.send(request);
    }

    public void deleteUser() throws Exception {
        HttpRequest request = restClient.requestBuilder(
            URI.create(userEndpoint + "/3"), Optional.empty()).DELETE().build();
        HttpResponse<String> response = restClient.send(request);
    }
}

```

Now, we'll use this class to consume the user-service API's CRUD operations by implementing the command-line runner class, as shown in the following code:

```

@Component
@ConditionalOnProperty(prefix = "command.autorun", name = "enabled", havingValue = "tr

```

```

class JavallHttpClientSample implements CommandLineRunner {

    private static final Logger LOG = LoggerFactory.getLogger(JavallHttpClientSample.class);

    // Java 11 HttpClient for calling User REST endpoints
    @Autowired
    private UserRestClient httpClient;

    @Override
    public void run(String... strings) throws Exception {
        LOG.info("Creating new user");
        httpClient.postUser();
        LOG.info("\n\n\nUpdate newly created user");
        httpClient.putUser();
        LOG.info("\n\nRetrieve users");
        httpClient.getUser();
        LOG.info("\n\n\nPatch updated user");
        httpClient.patchUser();
        LOG.info("\n\nRetrieve patched user");
        httpClient.getUser();
        LOG.info("\n\n\nDelete newly created users");
        httpClient.deleteUser();
        LOG.info("\n\nRetrieve user again
                to check if deleted user still exists");
        httpClient.getUser();
    }
}

```

[Support](#) [Sign Out](#)